

# Trabalho 1 - Análise de complexidade em estruturas de listas

**Grupo:** Kaio Henrique da Silva Nezio, Thiago Fabiano e Daniel Pinheiro

## Seção 1 - Relato sobre o desenvolvimento da biblioteca e do programa de teste

No desenvolvimento desse projeto, cada integrante do grupo teve participação direta na sua elaboração e execução. Antes de darmos início de fato, estruturamos nosso projeto no github para deixar tudo mais organizado e mais fácil de compreender. Logo após isso, dividimos o trabalho em três frentes, e ao longo do desenvolvimento cada um também apoiou as frentes dos outros. Segue abaixo o que cada integrante desenvolveu:

- **Daniel Pinheiro:** montou a estrutura inicial do projeto no repositório, a partir do código base disponibilizado pelo professor, e criou o esqueleto das classes do programa de contatos. Escreveu o README com a organização do código e as instruções de execução, retirou da entrega os arquivos de exemplo do professor, atuou na construção e revisão do relatório e realizou o envio do relatório.
- **Thiago Fabiano:** criou o repositório no github e propôs a divisão inicial das etapas. Implementou a biblioteca de lista encadeada genérica: o método adicionar nas duas versões (listaordenada e não ordenada), o quantidadeNos e o toString. Também desenvolveu o programa gerador dos arquivos de entrada usados nos testes de desempenho. No relatório, escreveu a análise matemática de complexidade do adicionar e do quantidadeNos. Participou dos testes finais do programa e dos ajustes que surgiram. Com o código já na versão final, repetiu as medições da Seção 3, criou os gráficos e fez ajustes em suas interpretações.
- **Kaio Henrique da Silva Nezio:** implementou os métodos pesquisar e remover da lista, incluindo a parada antecipada da busca quando a lista está ordenada, e desenvolveu o programa de contatos que utiliza a biblioteca (classe Contato, os comparadores por nome e por telefone e o menu interativo). Executou a bateria de testes empíricos com os arquivos de diferentes tamanhos. No relatório, escreveu a análise matemática do pesquisar e do remover, a Seção 1 e a Seção 3.

Além da sua própria frente, todos os integrantes participaram dos testes do programa e da revisão do código e do relatório dos demais.

Durante o desenvolvimento deste trabalho, ferramentas de IA foram adotadas com a finalidade de apoio técnico, revisão e pesquisa. A estruturação lógica da biblioteca ListaEncadeada, a escolha das estruturas de dados e a condução dos testes empíricos foram idealizadas e executadas pelos componentes do grupo.

De forma mais detalhada, a IA foi utilizada para:

- A ferramenta revisou a lógica de manipulação de ponteiros nos métodos 'pesquisar' e 'remover', garantindo que casos de borda (como a remoção do último elemento) não gerassem exceções de memória. Serviu também para consultas sobre boas práticas de sintaxe na leitura de arquivos via `BufferedReader`;
- A IA auxiliou na divisão de trabalho entre os integrantes, sugerindo uma divisão equivalente e paralelizável;
- No relatório, ela agiu como revisora textual, auxiliando na formatação das análises matemáticas de complexidade, e na elaboração das tabelas e gráficos, os testes foram feitos manualmente, entregamos em texto simples os resultados e a IA gerou as tabelas e imagens dos gráficos.

Todos os arquivos do projeto podem ser acessados pelo seguinte link do github: <https://github.com/ThiagoFabiano/bsi-tpa-trabalhos>. Nele está localizado um README que explica a organização das pastas, o formato do arquivo de entrada e como compilar e executar o projeto.

## **Seção 2 – Análise de complexidade dos métodos adicionar, quantidadeNos, pesquisar e remover**

Para medir o tempo dos métodos, contamos quantas vezes cada linha do código roda. Esse tempo é escrito como  $T(n)$ , e o  $n$  é o número de contatos que já estão na lista na hora em que o método é chamado.

As regras que usamos são:

- comando simples (uma atribuição, um teste, uma conta) gasta sempre o mesmo tempo, então é  $O(1)$ ;
- vários comandos simples um atrás do outro continuam sendo  $O(1)$ ;
- no if-else só um dos lados roda, então a gente considera o lado mais demorado;
- no laço, o tempo é o número de voltas multiplicado pelo tempo de uma volta.

Depois de somar tudo, ficamos só com a parte que cresce mais rápido e descartamos o resto, igual ao exemplo: se  $T(n) = 8n^3 + 4n - 10$ , a resposta é  $O(n^3)$ , porque o 8, o  $4n$  e o  $-10$  quase não pesam quando  $n$  fica grande.

### **2.1 O que a lista guarda**

Três atributos da ListaEncadeada aparecem o tempo todo nas contas abaixo: o `prim`, que aponta para o primeiro nó; o `ult`, que aponta para o último; e o `quant`, que é um

número inteiro com a quantidade de nós. Além deles, a lista recebe no construtor o Comparator (nas nossas listas, um que compara contatos pelo nome e outro que compara pelo telefone) e um boolean dizendo se ela vai ser ordenada.

## 2.2 Método adicionar

```
1 public boolean adicionar(T novoValor) {  
2     if (this.ehOrdenada) {  
3         adicionarOrdenado(novoValor);  
4     } else {  
5         adicionarNaoOrdenado(novoValor);  
6     }  
7     this.quant++;  
8     return true;  
9 }
```

*Listagem 1 – método adicionar*

O adicionar quase não faz trabalho nenhum. Ele testa o ehOrdenada na linha 2, chama um dos dois métodos de inserção, soma 1 no contador na linha 7 e devolve true na linha 8. O retorno é sempre true porque a lista aceita valor repetido: quem proíbe dois contatos com o mesmo telefone é o menu do programa, não a lista.

Tirando as chamadas das linhas 3 e 5, todas as linhas são comandos simples. Como as duas chamadas estão num if-else, o tempo do adicionar vai ser o do mais demorado dos dois. Então precisamos olhar cada um.

### 2.2.1 Quando a lista não é ordenada

```
1 private void adicionarNaoOrdenado(T novoValor) {  
2     No<T> novo = new No<>(novoValor);  
3     if (this.prim == null) {  
4         this.prim = novo;  
5         this.ult = novo;  
6     } else {  
7         this.ult.setProx(novo);  
8         this.ult = novo;  
9     }  
10 }
```

*Listagem 2 – inserção quando a lista não é ordenada*

A conta aqui é fácil, porque não tem laço. A linha 2 cria o nó e a linha 3 testa se a lista está vazia. Se estiver, as linhas 4 e 5 fazem o nó novo ser o primeiro e o último ao mesmo tempo. Se não estiver, as linhas 7 e 8 penduram o nó novo no fim e guardam ele como o novo ult. São duas atribuições de um lado ou do outro, nunca os dois.

Somando, o tempo é sempre o mesmo valor fixo:

**$T(n) = c$** , então o método é  $O(1)$  (constante)

Com a lista vazia ou com 400.000 contatos dentro dela, esse método executa o mesmo número de linhas. Ele consegue isso porque a lista guarda o ult. Se guardasse só o prim, para achar o fim seria preciso passar por todos os nós e o método seria  $O(n)$ .

### 2.2.2 Quando a lista é ordenada

```
1 private void adicionarOrdenado(T novoValor) {
2     No<T> novo = new No<>(novoValor);
3     if (this.prim == null) {
4         this.prim = novo;
5         this.ult = novo;
6         return;
7     }
8     No<T> ant = null;
9     No<T> atual = this.prim;
10    while (atual != null &&
11        this.comparador.compare(atual.getValor(), novoValor) < 0) {
12        ant = atual;
13        atual = atual.getProx();
14    }
15    if (ant == null) {
16        novo.setProx(this.prim);
17        this.prim = novo;
18    } else if (atual == null) {
19        this.ult.setProx(novo);
20        this.ult = novo;
21    } else {
22        ant.setProx(novo);
23        novo.setProx(atual);
24    }
25 }
```

*Listagem 3 – inserção quando a lista é ordenada*

Agora o contato não pode entrar em qualquer lugar. Antes de encaixar o nó, o método tem que descobrir onde é o lugar dele, e quem faz isso é o while das linhas 10 a 14. Ele anda pela lista enquanto o contato do nó atual for menor que o contato que está entrando, levando junto o ant para não perder o nó de trás. Quando o while para, as linhas 15 a 24 só amarram os ponteiros, e isso é rápido: cai em um dos três casos (entrar na frente de todo mundo, entrar no fim, ou entrar no meio) e cada um faz no máximo duas atribuições.

| Linha   | O que faz                                                  | Quantas vezes roda no pior caso |
|---------|------------------------------------------------------------|---------------------------------|
| 2 a 9   | cria o nó, trata a lista vazia e prepara o ant e o atual   | 1                               |
| 10 e 11 | testa se dá para continuar andando (compara dois contatos) | n + 1                           |
| 12 e 13 | passa para o próximo nó                                    | n                               |
| 15 a 24 | encaixa o nó no lugar                                      | 1                               |

As linhas de fora do while dão um tempo fixo. O que pesa é o while, que no pior caso dá n voltas, cada uma com o mesmo custo:

$T(n) = c_1 \cdot n + c_2$ , que é uma conta do tipo  $an + b$ , então o método é  $O(n)$  (linear)

O pior caso é quando o contato que está entrando é maior que todos os que já estão na lista. O while só para quando o atual chega no null, depois de comparar com os  $n$  contatos. Isso acontece o tempo todo se o arquivo estiver em ordem alfabética, porque cada contato novo seria maior que todos os anteriores. Nos arquivos que geramos para os testes isso não acontece, porque o gerador embaralha as linhas antes de gravar, mas o pior caso continua valendo como limite: nenhuma inserção vai demorar mais que isso.

Se o contato for menor que o primeiro da lista, o while nem entra e o método gasta tempo fixo. Esse é o melhor caso.

### 2.3 Método quantidadeNos

```
1 public int quantidadeNos() {  
2     return this.quant;  
3 }
```

*Listagem 4 – método quantidadeNos*

Uma linha só, que devolve um número já pronto. É um comando simples e roda uma vez, então  $T(n) = c$  e o método é  $O(1)$ . Não tem pior caso: ele faz a mesma coisa sempre.

O que deixa ele constante é o quant ser atualizado por quem mexe na lista, o adicionar somando 1 (linha 7 da Listagem 1) e o remover tirando 1. Se a gente tivesse escolhido contar os nós na hora da pergunta, seria preciso um laço percorrendo a lista toda e o método seria  $O(n)$ . Isso apareceria na opção Sair do programa, que mostra o total de contatos cadastrados.

### 2.4 Diferença entre lista ordenada e lista não ordenada

Dos dois métodos analisados até aqui, só o adicionar muda:

| Método        | Lista não ordenada | Lista ordenada |
|---------------|--------------------|----------------|
| adicionar     | $O(1)$             | $O(n)$         |
| quantidadeNos | $O(1)$             | $O(1)$         |

A lista não ordenada não compara o contato novo com ninguém, só pendura ele no fim usando o ult. A ordenada tem que procurar o lugar certo, e essa procura é feita de um nó por vez, começando do primeiro.

Dá para prever com isso o tempo de carregar o arquivo, que é a primeira medida da Seção 3. Nossos arquivos de entrada têm uma linha por contato, no formato nome;telefone, e foram gerados com 50.000, 100.000, 200.000 e 400.000 contatos. Para montar a lista, o programa chama o adicionar uma vez por linha.

Na lista não ordenada, são  $n$  chamadas de tempo fixo, o que dá  $O(n)$ : dobrando o arquivo, o tempo dobra. Na ordenada, no pior caso, a primeira inserção compara com 0 contatos, a segunda com 1, a terceira com 2, e por aí vai, somando  $0 + 1 + 2 + \dots + (n - 1) = n \cdot (n - 1) / 2$ , que é  $O(n^2)$ . Aí dobrar o arquivo faz o tempo ficar umas quatro vezes

maior, e é por isso que esperamos ver o carregamento no modo ordenado disparar nos arquivos de 200.000 e 400.000 contatos.

Ordenar não deixa a inserção mais rápida em nenhum momento. Mesmo com a lista toda em ordem, não dá para pular direto para o meio dela, que é o que a pesquisa binária faz, porque numa lista encadeada só se chega em um nó passando por todos os que vêm antes.

## 2.5 Método pesquisar

```
1 public T pesquisar(T valor) {
2     if (valor == null || this.prim == null) {
3         return null;
4     }
5     No<T> atual = this.prim;
6     while (atual != null) {
7         int comp = this.comparador.compare(atual.getValor(), valor);
8         if (comp == 0) {
9             return atual.getValor();
10        }
11        if (this.ehOrdenada && comp > 0) {
12            return null;
13        }
14        atual = atual.getProx();
15    }
16    return null;
17 }
```

*Listagem 5 - método pesquisar*

O método 'pesquisar' precisa varrer a lista procurando um contato específico. Ele começa testando se a lista está vazia (linha 2) e cria um ponteiro 'atual' apontando para o início (linha 5). O trabalho pesado acontece no laço 'while' (linhas 6 a 15).

| Linha  | O que faz                                       | Quantas vezes roda no pior caso |
|--------|-------------------------------------------------|---------------------------------|
| 2 a 5  | Trata lista vazia e prepara o ponteiro inicial  | 1                               |
| 6      | Testa se chegou ao fim da lista (atual != null) | n+1                             |
| 7 a 13 | Compara os valores e testa condições de parada  | n                               |
| 14     | Passa para o próximo nó                         | n                               |
| 16     | Retorna null se não achou                       | 1                               |

Fora do laço, temos apenas comandos simples de tempo fixo. Dentro do laço, a cada volta, o método faz comparações e avança o ponteiro, o que custa  $O(1)$ . Como o laço pode dar até  $n$  voltas, o tempo total é do tipo  $T(n) = c_1 * n + c_2$ , o que nos dá uma complexidade  $O(n)$  (linear).

### Quando ocorre o pior caso?

O pior caso ocorre quando o elemento procurado **não existe** na lista ou é exatamente o **último elemento**. Nessas situações, o laço não consegue parar no meio (linhas 8 e 11) e é obrigado a percorrer todos os  $n$  contatos até que o 'atual' vire null.

### Diferença entre lista ordenada e não ordenada na pesquisa:

Matematicamente, ambas são  $O(n)$ . O que muda é que a lista ordenada possui uma "válvula de escape" na linha 11. Se estivermos procurando o nome "Carlos" e o ponteiro chegar em "Daniel", a lista ordenada sabe que o "Carlos" não está mais para frente ( $comp > 0$ ), abortando o laço mais cedo. A lista não ordenada, por não ter garantia de onde os itens estão, precisaria ir até o final. Isso não muda a ordem de complexidade (pois no pior caso de procurar o "Zico", a ordenada também vai até o final), mas na prática deixa a busca de itens inexistentes mais rápida.

## 2.6 Método remover

### 2.6 Método remover

```
1 @Override
2 public boolean remover(T valor) {
3     return remover(valor, this.comparador);
4 }
5
6 // Remove usando outro criterio de comparacao. Serve para quem usa a lista
7 // poder achar o elemento exato quando a lista esta ordenada por outro campo.
8 public boolean remover(T valor, Comparator<T> criterio) {
9     if (valor == null || this.prim == null){
10         return false;
11     }
12     No<T> ant = null;
13     No<T> atual = this.prim;
14     while (atual != null) {
15         int comp = criterio.compare(atual.getValor(), valor);
16         if(comp == 0){
17             // Caso 1: O elemento a ser removido é o primeiro da lista
18             if (ant == null){
19                 this.prim = atual.getProx();
20                 // Se a lista só tinha 1 elemento, o ult também deve virar null
21                 if (this.prim == null){
22                     this.ult = null;
23                 }
24             }
25         }
26         ant = atual;
27         atual = atual.getProx();
28     }
29 }
```

```

24     } else{
25         // Caso 2: O elemento está no meio ou no fim
26         ant.setProx(atual.getProx());
27         // Se removeu o último, atualiza o ponteiro 'ult' para o anterior
28         if (atual == this.ult){
29             this.ult = ant;
30         }
31     }
32     this.quant--;
33     return true;
34 }
35 ant = atual;
36 atual = atual.getProx();
37 }
38 return false;
39 }

```

*Listagem 6 - método remover*

O 'remover' é parecido com o 'pesquisar', mas guarda o ponteiro ant (linhas 12 e 35) para religar a lista quando o nó sai. Ele tem duas versões: a da linha 2, exigida pela interface IColecao, que usa o comparador da própria lista, e a da linha 8, que recebe outro critério. A segunda foi necessária porque muitos contatos têm o mesmo nome, e remover por nome apagaria o homônimo errado. Com o telefone, que é único, a remoção acerta o contato certo.

Quando ele encontra o 'contato' (comp == 0 na linha 16), as linhas de 17 a 33 apenas trocam as referências de memória para "pular" o nó removido e atualizam o contador this.quant. Ajustar ponteiros e fazer uma subtração custam apenas **O(1)**. Portanto, o que dita o tempo do método é a 'busca'.

Como ele usa a mesma lógica de percorrer a lista nó a nó, sua complexidade também é **O(n)**.

O pior caso acontece quando pedimos para remover o último 'contato' da lista, ou quando o contato não existe. O laço dará 'n' voltas até chegar no fim, gastando tempo proporcional ao tamanho da lista antes de realizar a remoção de O(1). Diferente do 'pesquisar', o 'remover' não tem a parada antecipada da lista ordenada. O motivo é o critério de comparação: como o método pode receber um critério diferente do que ordenou a lista (é o caso da lista de nomes, onde procuramos pelo telefone), não dá para garantir que o elemento não está mais à frente, então ele sempre percorre até achar ou até o fim. Assim, o 'remover' é O(n) na lista ordenada e na não ordenada, tanto para achar quanto para não achar o contato.

### Seção 3 - Análise Empírica de complexidade de Algoritmos da sua biblioteca

Usamos quatro tamanhos de arquivo: 50.000, 100.000, 200.000 e 400.000 registros. Todas as medições foram feitas na mesma máquina e com a mesma versão do código. Em cada execução carregamos o arquivo, pesquisamos o último elemento da lista pelo telefone e pelo nome, pesquisamos um nome inexistente e removemos esse mesmo elemento pelo telefone. Na lista não ordenada repetimos cada medição três vezes e usamos o valor do meio; na lista ordenada, uma execução por tamanho.

O último elemento da lista muda conforme o modo: na não ordenada é o último contato do arquivo, já que a inserção segue a ordem de chegada; na ordenada é o contato com o maior telefone, ou o maior nome, na lista de nomes. Nos dois casos é o elemento que força a busca a percorrer a lista inteira.

#### Tempo de montagem das listas

A primeira medida é o tempo para ler o arquivo de entrada e montar as duas listas.

| Tamanho | Não ordenada (ms) | Razão | Ordenada (s) | Razão |
|---------|-------------------|-------|--------------|-------|
| 50.000  | 43,4              | —     | 46,93        | —     |
| 100.000 | 91,1              | 2,10x | 205,99       | 4,39x |
| 200.000 | 128,2             | 1,41x | 886,21       | 4,30x |
| 400.000 | 171,8             | 1,34x | 4.176,89     | 4,71x |

Tabela 1 – Tempo de montagem das listas em função do tamanho da entrada

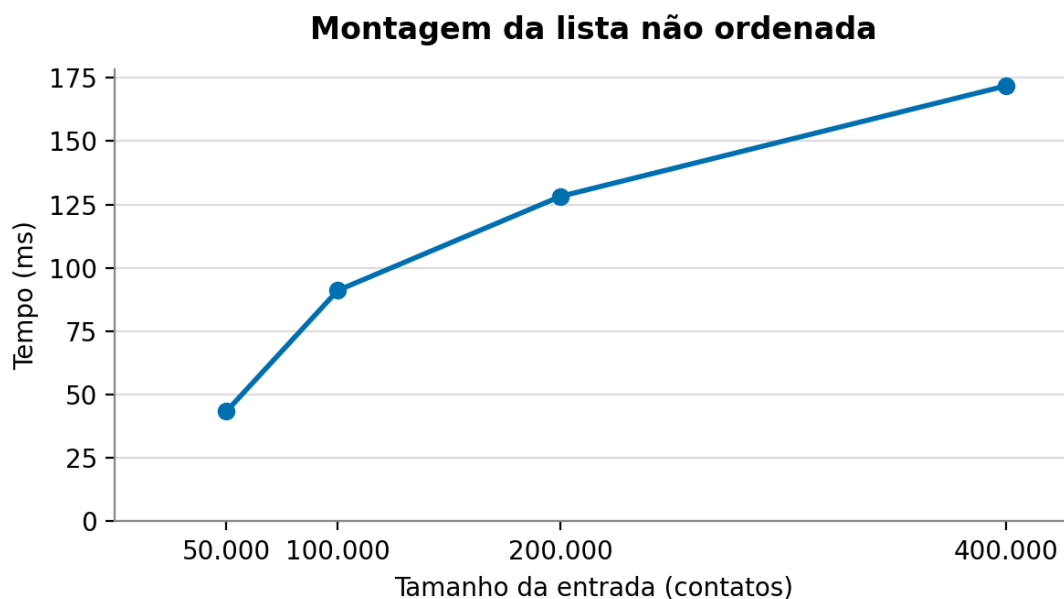


Gráfico 1 – Montagem da lista não ordenada

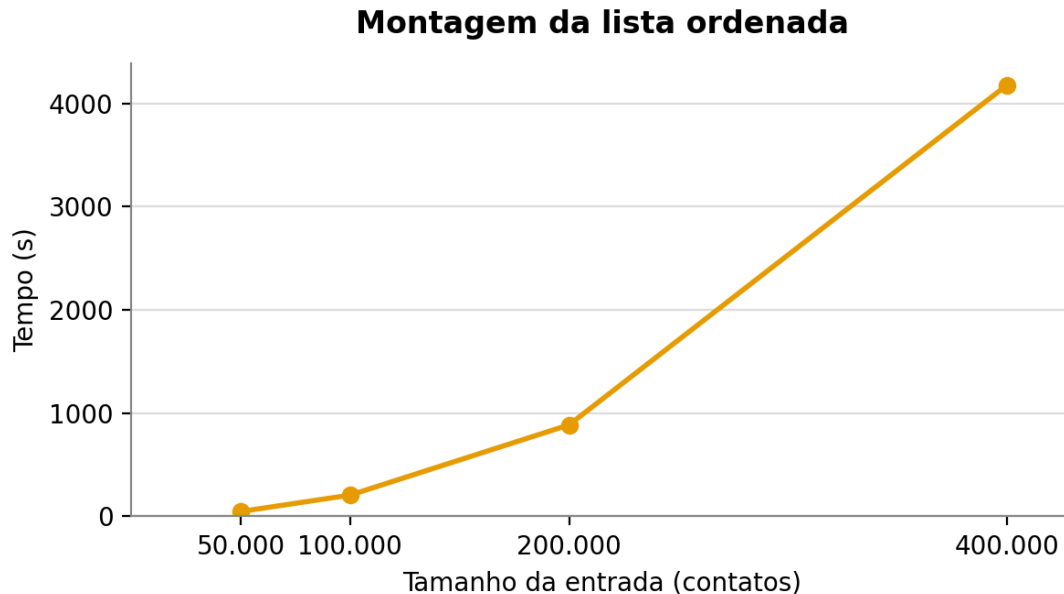


Gráfico 2 – Montagem da lista ordenada

Os dois gráficos estão separados porque as escalas são muito diferentes: a lista não ordenada trabalha na casa dos milissegundos e a ordenada na casa dos segundos. Montar 400.000 contatos sem ordenação levou menos de dois décimos de segundo; a mesma quantidade na lista ordenada levou 1 hora e 10 minutos.

### Pesquisa do último contato pelo telefone

| Tamanho | Não ordenada (ns) | Ordenada (ns) |
|---------|-------------------|---------------|
| 50.000  | 5.432.309         | 9.141.336     |
| 100.000 | 7.446.970         | 10.828.974    |
| 200.000 | 8.097.667         | 17.345.698    |
| 400.000 | 10.779.846        | 34.486.922    |

Tabela 2 – Tempo de pesquisa pelo telefone em função do tamanho da entrada

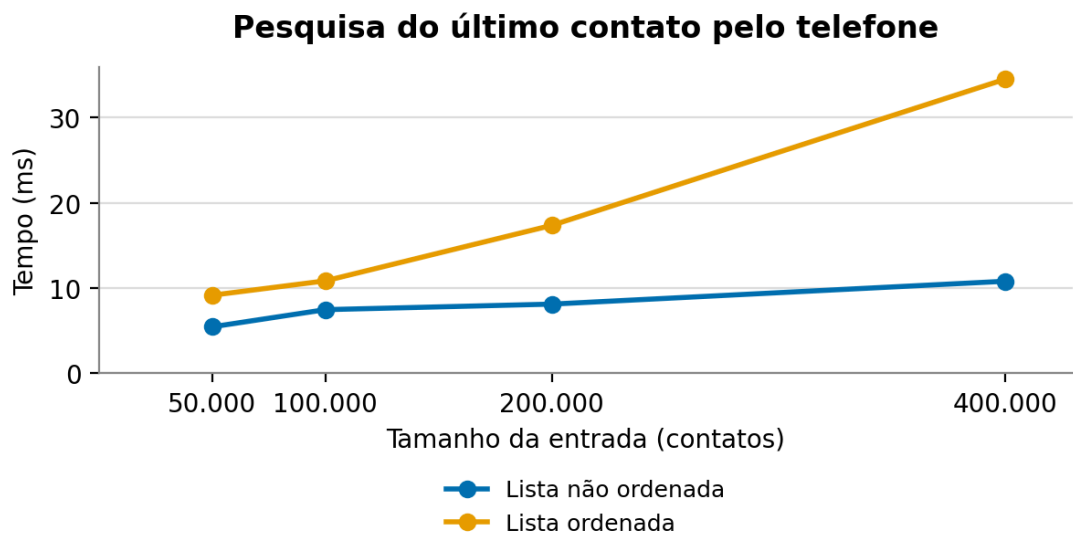


Gráfico 3 – Pesquisa do último contato pelo telefone

O telefone não se repete em nenhum contato, então a busca só termina quando encontra exatamente o elemento procurado. Como ele é o último da lista, o método percorre todos os contatos, e é por isso que o tempo cresce junto com o tamanho da entrada.

### Pesquisa pelo nome

| Tamanho | Último contato – não ordenada (ns) | Nome inexistente – não ordenada (ns) | Último contato – ordenada (ns) | Nome inexistente – ordenada (ns) |
|---------|------------------------------------|--------------------------------------|--------------------------------|----------------------------------|
| 50.000  | 429.517                            | 3.284.385                            | 6.129.911                      | 5.142.347                        |
| 100.000 | 222.734                            | 3.153.977                            | 5.337.331                      | 4.031.668                        |
| 200.000 | 325.268                            | 4.584.537                            | 10.595.129                     | 6.966.316                        |
| 400.000 | 409.859                            | 7.488.549                            | 22.615.372                     | 17.412.152                       |

Tabela 3 – Tempo de pesquisa pelo nome em função do tamanho da entrada

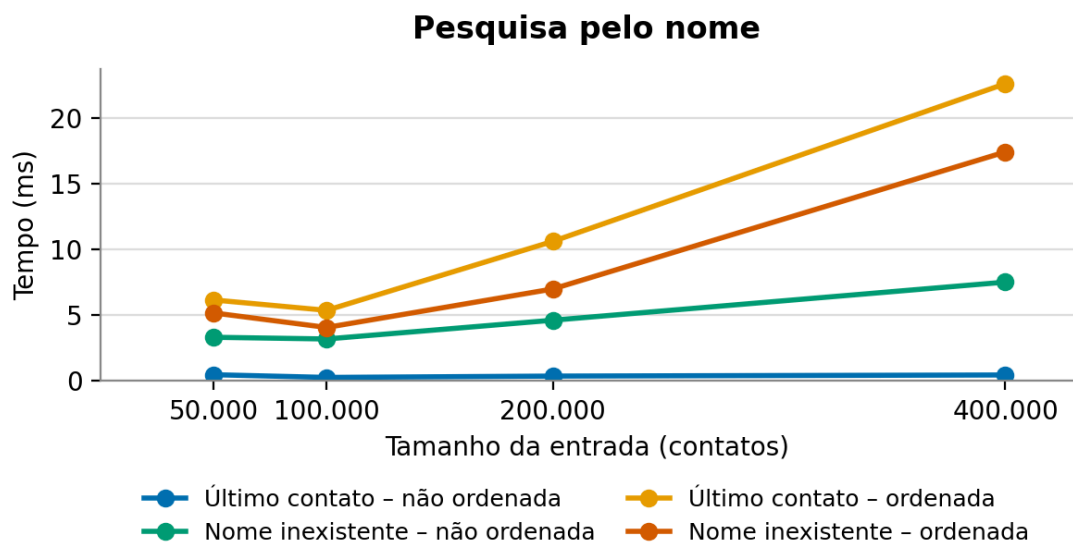


Gráfico 4 – Pesquisa pelo nome

Os tempos da busca pelo nome do último contato quase não crescem: 429.517 ns com 50.000 contatos e 409.859 ns com 400.000. O motivo está no nosso gerador de arquivos, que monta os nomes combinando 22 primeiros nomes com 20 sobrenomes, ou seja, só existem 440 nomes diferentes. Com 400.000 contatos cada nome se repete centenas de vezes, e o primeiro contato com aquele nome aparece logo nas primeiras linhas do arquivo. Como o método pesquisar devolve o primeiro que encontrar, o que está correto segundo a especificação, a busca para quase no começo da lista e não representa o pior caso.

Para medir o pior caso de verdade, repetimos a busca procurando um nome que não existe na lista, o que obriga o método a percorrer todos os  $n$  contatos. Essa coluna mostra o comportamento esperado, crescendo junto com o tamanho da entrada. Na lista ordenada isso não é necessário, porque os homônimos ficam agrupados no fim e o nome procurado é o maior de todos, então a busca já percorre a lista quase inteira.

### Remoção do último contato pelo telefone

| Tamanho | Não ordenada (ns) | Ordenada (ns) |
|---------|-------------------|---------------|
| 50.000  | 4.269.648         | 9.174.849     |
| 100.000 | 6.034.364         | 9.702.087     |
| 200.000 | 7.338.313         | 16.827.141    |
| 400.000 | 10.043.767        | 35.195.429    |

Tabela 4 – Tempo de remoção em função do tamanho da entrada

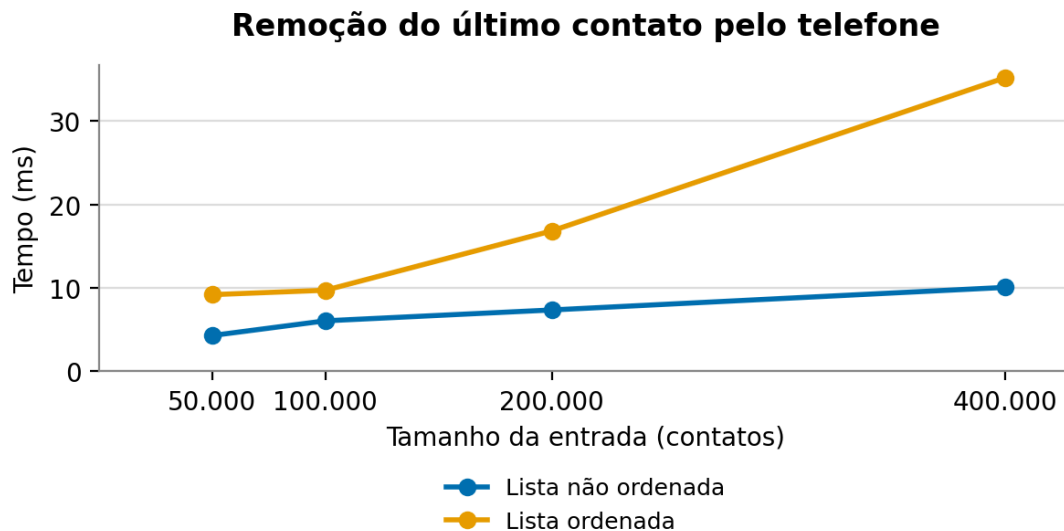


Gráfico 5 – Remoção do último contato pelo telefone

O tempo apresentado é o da chamada ao método `remove` na lista indexada por telefone, que é o que a especificação pede. A remoção do mesmo contato na lista indexada por nome acontece em seguida, fora do cronômetro, e também é  $O(n)$ . Os tempos são parecidos com os da pesquisa porque `remove` é, antes de tudo, procurar: o método percorre a lista até achar o contato e só então religa os ponteiros, o que custa tempo fixo.

## Os tempos batem com a análise da Seção 2?

Na Seção 2 concluímos que o adicionar é  $O(1)$  na lista não ordenada e  $O(n)$  na ordenada, e que por isso montar uma lista com  $n$  contatos deveria custar  $O(n)$  no primeiro caso e  $O(n^2)$  no segundo. É o que aparece na Tabela 1.

Na lista ordenada, a previsão era que dobrar a entrada multiplicasse o tempo por quatro. Foi o que aconteceu nos três saltos: de 50.000 para 100.000 o tempo passou de 46,93 s para 205,99 s, ou 4,39 vezes; de 100.000 para 200.000 passou para 886,21 s, ou 4,30 vezes; e de 200.000 para 400.000 chegou a 4.176,89 s, ou 4,71 vezes. Os três valores ficam logo acima de 4, o que também era esperado: além do número de comparações crescer ao quadrado, a lista grande não cabe mais na memória rápida do processador, e cada passo de um nó para o outro passa a custar um pouco mais caro.

Na lista não ordenada, a previsão era crescimento linear. O tempo saiu de 43,4 ms para 171,8 ms enquanto a entrada ficou oito vezes maior, ou seja, cresceu devagar e sem nenhuma explosão. As razões medidas foram 2,10, 1,41 e 1,34: a primeira bate com o dobro esperado e as outras duas ficam abaixo. Isso acontece porque o tempo cronometrado não é só o do adicionar, ele inclui ler o arquivo do disco, separar o nome do telefone e criar os objetos. Existe também um custo fixo de aquecimento da máquina virtual do Java, que só otimiza o código depois de executá-lo alguns milhares de vezes, e esse custo pesa muito no arquivo pequeno e quase nada no grande.

A pesquisa e a remoção são  $O(n)$  nos dois tipos de lista, e é isso que as Tabelas 2, 3 e 4 mostram: os tempos crescem junto com o tamanho da entrada, mas ficam todos na casa dos milissegundos, porque percorrer a lista uma vez é muito mais barato do que percorrê-la a cada inserção.

## **Conclusão**

Como podemos visualizar, o maior problema da lista ordenada foi no momento de carregar o arquivo. Como o programa precisa percorrer os nós para achar a posição certa a cada contato inserido, o tempo cresce muito rápido: com 400.000 contatos a montagem levou 1 hora e 10 minutos. Já na lista não ordenada a inserção é  $O(1)$ , jogando o contato direto no fim, e a mesma quantidade de contatos foi montada em menos de dois décimos de segundo.

Com isso tudo, é evidente que a ordenação das listas telefônicas não compensou em nenhum dos casos, demonstrando que o uso de listas não ordenadas é mais eficaz para o tratamento desses dados.